Sicherheitsbericht Adressverwaltung

Stand: 10. Oktober 2026 · Version 2.5.0 · Frontend (Next.js), Backend (ASP.NET Core OData API), nginx, Docker Compose, Hilfsskripte

1. Zusammenfassung

Der Bericht umfasst zwei Prüfungen. Die erste vom 4. Oktober 2026 ergab 18 Befunde (S-01 bis S-18): 2 kritische, 6 hohe, 5 mittlere und 5 niedrige. Die zweite vom 9. Oktober 2026 ergab 11 weitere (S-19 bis S-29): 7 mittlere und 4 niedrige, keine kritischen oder hohen. Anschliessend wurden am selben Tag die bis dahin offenen Restrisiken O-03 bis O-08 bearbeitet (Abschnitt 3.3).

Von den 29 Befunden sind 28 im Code behoben und am laufenden Stack überprüft. Einer ist nur im Code behoben: Das bisherige Standardpasswort des Admin-Benutzers gilt in der bestehenden Datenbank weiterhin (S-05); das ist bewusst akzeptiert.

Die zwei kritischen Befunde erlaubten den Zugriff auf alle Daten ohne Anmeldung: Jeder konnte sich über `/auth/register` selbst ein Konto anlegen, und die eingesetzte Next.js-Version liess sich am Seitenschutz vorbei aufrufen.

Die Adressverwaltung ist eine Demo-Anwendung ohne schützenswerte Daten. Deshalb sind zwei Punkte bewusst akzeptiert (Entscheid vom 4. Oktober 2026): die Geheimnisse in der Git-History auf GitHub und das alte Admin-Passwort in der bestehenden Datenbank. Von den sechs Restrisiken O-03 bis O-08 sind fünf behoben. O-07 ist verkleinert: Von 9 Meldungen zu Entwicklungswerkzeugen bleiben 5, für die es keine korrigierte Version gibt. Was darüber hinaus offen bleibt, steht in Abschnitt 4.

2. Umfang und Vorgehen

- Manuelle Durchsicht des Quellcodes: Controller, DbContext, Seeder, DTOs, `Program.cs`, `lib/auth.ts`, `lib/api.ts`, `middleware.ts`, Login-Formular, `nginx.conf`, `docker-compose.yml`, Dockerfiles, Skripte in `scripts/` und `migration/`.
- Abhängigkeiten: `dotnet list package --vulnerable --include-transitive` und `npm audit`.
- Überprüfung der Korrekturen: 51 Backend-Integrationstests (davon 5 neu für diese Korrekturen) und Aufrufe gegen den neu gebauten Docker-Stack unter https://localhost.
- Zweite Prüfung (9. Oktober 2026): erneute Durchsicht derselben Dateien und beider Abhängigkeitsprüfungen. Zusätzlich Aufrufe ohne Anmeldung gegen den laufenden Stack (Antwort-Header, erreichbare Endpunkte, angebotene TLS-Verfahren) und Anfragen mit unerwarteten Feldern an die OData-API über das Testprojekt. Überprüfung der Korrekturen mit 60 Tests (davon 9 neu) und am neu gebauten Stack.
- Bearbeitung der offenen Punkte (9. Oktober 2026): Überprüfung mit 69 Tests (davon 9 neu), am neu gebauten Stack und erstmals in einem Browser (Chromium, ferngesteuert mit Playwright) – Anmeldung, alle Seiten, Anlegen, Ändern, Löschen, Suchen, Blättern und Abmelden, je mit Auswertung der Browser-Konsole.

Nicht Teil der Prüfung: automatisierte Penetrationstests mit externen Werkzeugen, die Inhalte der Git-History im Detail und die Infrastruktur ausserhalb des Docker-Stacks.

3. Befunde und Korrekturen

3.1 Erste Prüfung (4. Oktober 2026)

Nr.	Stufe	Befund	Korrektur	Status
S-01	Kritisch	Offene Registrierung: `POST /auth/register` war ohne Anmeldung erreichbar. Ein neu angelegtes Konto erhält ein gültiges Token und damit vollen Lese- und Schreibzugriff auf alle Adressen, Städte und Einstellungen.	`[Authorize]` auf der Registrierung: Nur angemeldete Benutzer können weitere Benutzer anlegen. Die Tests legen ihren Benutzer direkt in der Datenbank an.	Behoben
S-02	Kritisch	Next.js 14.2.5 mit bekannten Lücken, darunter die Umgehung der Middleware (GHSA-f82v-jwr5-mffw), die hier den gesamten Seitenschutz bildet, und Codeausführung über die Bildoptimierung (GHSA-2xp9-vwfh-vxw4). Die 14er-Linie erhält dafür keine Korrekturen mehr.	Upgrade auf Next.js 15.5.27 und React 19. `npm audit --omit=dev` meldet keine Lücken mehr. PostCSS per `overrides` auf eine korrigierte Version angehoben.	Behoben
S-03	Hoch	Geheimnisse im Repository: JWT-Schlüssel, NextAuth-Secret und Datenbankpasswort standen im Klartext in `docker-compose.yml`. Mit dem JWT-Schlüssel lassen sich gültige Tokens für beliebige Benutzer fälschen.	Geheimnisse in die Datei `.env` ausgelagert (git-ignoriert, Vorlage `.env.example`); `docker-compose.yml` setzt sie als Variablen ein. Alle drei Werte neu erzeugt; das Datenbankpasswort wurde in der laufenden Datenbank geändert.	Behoben (History: O-01, akzeptiert)
S-04	Hoch	Platzhalter-JWT-Schlüssel in `appsettings.json`, der ohne eigene Konfiguration stillschweigend verwendet wurde; dazu das Datenbankpasswort in derselben Datei.	Schlüssel und Passwort aus `appsettings.json` entfernt. Das Backend startet nicht, wenn `Jwt:Key` fehlt, kürzer als 32 Zeichen oder der alte Platzhalter ist.	Behoben
S-05	Hoch	Festes Standardpasswort für `admin@example.com` im Quellcode (`DbSeeder.cs`) und im README. Compose setzte kein eigenes Passwort, der Stack lief also mit diesem bekannten Zugang.	Kein Standardpasswort mehr: Ohne `Seed:AdminPassword` erzeugt das Backend ein Zufallspasswort und schreibt es einmalig ins Log. Das gilt nur für eine leere Datenbank – in der bestehenden funktioniert das alte Passwort weiterhin (geprüft).	Code behoben; Altpasswort akzeptiert (O-02)
S-06	Hoch	PostgreSQL (5432) und Backend (5000, ohne TLS) waren auf allen Netzwerkschnittstellen des Hosts veröffentlicht und damit am nginx vorbei erreichbar.	Beide Ports nur noch an `127.0.0.1` gebunden. Aus dem Netzwerk ist ausschliesslich nginx (80/443) erreichbar.	Behoben
S-07	Hoch	Kein Schutz gegen Passwort-Raten: Anmeldeversuche waren unbegrenzt möglich.	nginx begrenzt `/auth/` und `/api/auth/callback/credentials` auf 10 Anfragen pro Minute und Client-IP (Antwort 429). Im Test wurden 21 von 25 schnellen Versuchen abgewiesen.	Behoben
S-08	Hoch	NuGet-Pakete mit bekannten Lücken (transitiv): `Microsoft.Extensions.Caching.Memory`,	Pakete aktualisiert, danach Migration auf .NET 10. Die Prüfung	Behoben

		`System.Text.Json` (hoch), `System.IdentityModel.Tokens.Jwt`, `Microsoft.IdentityModel.JsonWebTokens` (mittel).	meldet für beide Projekte keine anfälligen Pakete.	
S-09	Mittel	Open Redirect: Das Login-Formular leitete nach der Anmeldung ungeprüft auf den Parameter `callbackUrl` weiter, also auch auf fremde Seiten (Phishing über einen echten Login-Link).	Nur noch Pfade innerhalb der eigenen App werden akzeptiert, alles andere führt auf die Startseite.	Behoben
S-10	Mittel	Audit-Felder fälschbar: `CreatedBy`/`ChangedBy` wurden ersatzweise aus dem frei setzbaren Header `X-User` übernommen.	Der Header wird nicht mehr gelesen; massgebend ist allein der Name aus dem Token. Durch einen Test abgedeckt.	Behoben
S-11	Mittel	Benutzer-Enumeration: Der Login brauchte für unbekannte E-Mail-Adressen messbar länger (zusätzliche Hash-Berechnung), und die Registrierung antwortete für bestehende und neue Adressen unterschiedlich.	Fester, einmalig berechneter Vergleichs-Hash für unbekannte Benutzer; identische Antwort der Registrierung in beiden Fällen.	Behoben
S-12	Mittel	Backend- und Frontend-Container liefen als `root`.	`USER` in beiden Dockerfiles gesetzt (Backend: `app`, Frontend: `node`).	Behoben
S-13	Mittel	Frontend-Build ohne Lockfile: `npm install` holte bei jedem Build die jeweils neusten passenden Versionen, ohne Integritätsprüfung.	`package-lock.json` versioniert, Dockerfile verwendet `npm ci`.	Behoben
S-14	Niedrig	Unvollständige Sicherheitsheader; nginx gab seine Version preis.	Ergänzt: `Referrer-Policy`, `Permissions-Policy`, `Content-Security-Policy` (`frame-ancestors`, `base-uri`, `form-action`, `object-src`); bestehende Header auch auf Fehlerantworten; `server_tokens off`.	Behoben
S-15	Niedrig	Skripte `create_user.sh` und `reset_password.sh`: Eingaben wurden direkt in SQL und in Python-Quelltext eingesetzt, das Passwort am Ende im Klartext ausgegeben, Mindestlänge 6 statt 8.	Werte gehen als psql-Variablen bzw. über die Umgebung; keine Passwortausgabe; Mindestlänge 8. Mit einem Passwort und Namen voller Sonderzeichen getestet.	Behoben
S-16	Niedrig	PLZ-Suche: Ein Hochkomma in der Eingabe konnte den OData-Filter verändern (ohne Rechteausweitung).	Hochkommas werden verdoppelt.	Behoben
S-17	Niedrig	Keine Längenbegrenzung für E-Mail und Passwort bei Login und Registrierung.	`MaxLength` 256 bzw. 128 auf den DTOs.	Behoben
S-18	Niedrig	Ein echt wirkendes Passwort stand als «Standard» im Kommentar von `migration/import_cities.py`.	Aus der Datei entfernt.	Behoben (History: O-01, akzeptiert)

3.2 Zweite Prüfung (9. Oktober 2026)

Nr.	Stufe	Befund	Korrektur	Status

S-19	Mittel	Over-Posting bei `POST /odata/*`: Eine mitgeschickte `id` wurde übernommen und kollidierte später mit der Sequenz der Datenbank (Fehler 500 beim Anlegen weiterer Einträge). `changedBy` und `changeDate` liessen sich frei setzen, die Audit-Felder also fälschen.	Der Schlüssel wird vor dem Speichern zurückgesetzt, die Datenbank vergibt ihn. Änderungsangaben werden beim Anlegen geleert. Durch einen Test abgedeckt.	Behoben
S-20	Mittel	`PATCH` mit `id` im Body löste eine unbehandelte Ausnahme aus (Fehler 500).	Der Schlüssel wird beim Patchen ignoriert; die übrigen Felder werden übernommen. Durch einen Test abgedeckt.	Behoben
S-21	Mittel	OData-Fehlerantworten enthielten unter `innererror` den Ausnahmetyp und den Stacktrace, zum Beispiel bei `$top=1000`.	Neuer Filter `ODataErrorDetailFilter` entfernt diese Angaben; die Fehlermeldung bleibt. Durch einen Test abgedeckt.	Behoben
S-22	Mittel	Keine Kontosperre (bisher Teil von O-06): Die Begrenzung in nginx (S-07) gilt pro Client-IP. Verteilt über viele Adressen liess sich ein Passwort weiterhin unbegrenzt raten.	Nach 3 Fehlversuchen ist die E-Mail-Adresse 5 Minuten gesperrt (Antwort 429; bis zum 10. Oktober 2026 nach 5 Fehlversuchen für 15 Minuten), auch für das richtige Passwort. Gezählt wird ebenso für unbekannte Adressen, die Sperre verrät also nicht, welche Konten es gibt. Eine erfolgreiche Anmeldung setzt den Zähler zurück. Seit dem 10. Oktober 2026 setzt die Sperre beim betroffenen Benutzer zusätzlich ein Sperrkennzeichen in der Datenbank (gilt auch nach einem Neustart) und meldet sich per E-Mail bei der Adresse aus den Einstellungen. Durch Tests abgedeckt.	Behoben
S-23	Mittel	TLS 1.2 akzeptierte schwache Verfahren wie `AES256-SHA`: statisches RSA ohne Forward Secrecy, CBC und SHA-1.	Für TLS 1.2 nur noch ECDHE mit AES-GCM oder ChaCha20-Poly1305; Session-Tickets abgeschaltet. TLS 1.3 ist unverändert.	Behoben
S-24	Mittel	Das Frontend-Image lief auf Node.js 20. Diese Linie erhält seit dem 30. April 2026 keine Sicherheitsupdates mehr.	Umstellung aller drei Build-Stufen auf `node:22-alpine`.	Behoben
S-25	Mittel	Google-Anmeldung (derzeit nicht konfiguriert): Mit gesetzten Google-Variablen hätte jedes beliebige Google-Konto eine Session erhalten und die Seiten erreicht. Daten wären nicht sichtbar gewesen, weil dieser Session das Backend-Token fehlt.	NextAuth lehnt Anmeldungen ohne Backend-Token ab (Rückkehr zur Login-Seite mit Hinweis), und die Middleware lässt nur Sessions mit Backend-Token durch.	Behoben
S-26	Niedrig	`/odata` und `/odata/$metadata` waren ohne Anmeldung lesbar und legten das Datenmodell offen. Ein neuer Controller ohne `[Authorize]` wäre offen gewesen.	Fallback-Policy in `Program.cs`: Jeder Endpunkt verlangt ein gültiges Token. Einzige Ausnahme ist `POST /auth/login` mit `[AllowAnonymous]`. Durch Tests abgedeckt.	Behoben
S-27	Niedrig	API-Antworten mit Personendaten und Tokens waren	`Cache-Control: no-store` auf allen Backend-Antworten; beide Kennungen abgeschaltet; nur noch	Behoben

		zwischenspeicherbar. Die Antworten gaben `Server: Kestrel` und `X-Powered-By: Next.js` preis. Das Backend akzeptierte jedes HMAC-Verfahren für die Token-Signatur.	HS256 zugelassen.	
S-28	Niedrig	Die Content-Security-Policy beschränkte nicht, woher Skripte geladen und wohin Verbindungen aufgebaut werden. Backend- und Frontend-Container liefen mit den Standard-Capabilities von Docker.	CSP um `default-src 'self'` sowie `script-src`, `style-src`, `img-src` und `connect-src` ergänzt. Inline-Skripte bleiben erlaubt, weil Next.js sie braucht (siehe O-04). Beide Container laufen mit `no-new-privileges` und ohne Capabilities.	Behoben
S-29	Niedrig	`reset_password.sh` prüfte die Mindestlänge nicht, wenn das Passwort als Argument übergeben wurde. `create_user.sh` liess dieselbe E-Mail-Adresse in anderer Gross-/Kleinschreibung ein zweites Mal zu.	Die Längenprüfung gilt für beide Eingabewege. E-Mail-Adressen werden wie im Backend in Kleinbuchstaben gespeichert und ohne Rücksicht auf die Schreibweise verglichen.	Behoben

3.3 Bearbeitung der offenen Punkte (9. Oktober 2026)

Nr.	Ausgangslage	Korrektur	Status
O-03	OData-Listen ohne Seitengrösse: Eine Abfrage lieferte alle rund 4000 Städte in einer Antwort (4,3 MB für die Städteseite).	Das Backend liefert höchstens 100 Zeilen pro Antwort und einen Folgelink (`@odata.nextLink`). Die Listen im Frontend blättern mit 25 Zeilen pro Seite; die Städteliste hat dazu eine Suche nach PLZ oder Ortsname.	Behoben
O-04	Das Backend-Token war für JavaScript im Browser lesbar (`session.accessToken`); Inline-Skripte waren erlaubt.	Das Token bleibt auf dem Server: Der Browser ruft die API über `/api/backend` der eigenen Anwendung auf, die das Token aus dem verschlüsselten Cookie anhängt. `/api/auth/session` liefert kein Token mehr. Weil diese Aufrufe über ein Cookie angemeldet sind, verlangt der Proxy einen eigenen Header und einen passenden `Origin` (Schutz vor Cross-Site-Request-Forgery). Die CSP der Seiten erlaubt Skripte nur noch mit einer Nonce, die für eine einzige Antwort gilt.	Behoben
O-05	Keine Rollen: Jeder angemeldete Benutzer durfte alles.	Rollen `Admin` und `User`. Nur `Admin` darf Benutzer anlegen (`POST /auth/register`) und die Einstellungen ändern (`PUT /settings`); `User` pflegt Adressen und Städte. Die Rolle wird bei jeder Anfrage aus der Datenbank gelesen, ein Wechsel gilt also sofort. Bestehende Benutzer sind `Admin` geblieben, neue erhalten `User`.	Behoben
O-06	Kein Widerruf ausgestellter Tokens (8 Stunden gültig).	Jeder Benutzer hat eine Token-Version, die im Token steht und bei jeder Anfrage verglichen wird. Die Abmeldung (`POST /auth/logout`, von NextAuth automatisch aufgerufen) und `reset_password.sh` erhöhen sie; ältere Tokens sind damit ungültig. Tokens gelöschter Benutzer werden ebenfalls abgewiesen.	Behoben
O-07	`npm audit` meldete 9 Einträge in Entwicklungswerkzeugen	Migration auf Tailwind CSS 4; damit entfallen 4 Einträge. Es bleiben 5 (hoch), alle über `braces` im Lint-Plugin von Next.js. Für	Teilweise behoben

	(Tailwind CSS 3, ESLint).	`braces` gibt es keine korrigierte Version.	
O-08	Das Backend verband sich als Datenbank-Superuser `postgres`.	Eigene Rolle `bandaddress_app` ohne Superuser-Rechte, die nur die Datenbank der Anwendung besitzt. Neue Datenbanken erhalten sie beim ersten Start, bestehende über `./scripts/create_db_role.sh`. Neues Geheimnis `APP_DB_PASSWORD` in `.env`.	Behoben

4. Akzeptierte Risiken und offene Punkte

O-01 und O-02 sind für die Demo-Anwendung akzeptiert. Bei einem Einsatz mit echten Daten sind sie vor der Inbetriebnahme zu erledigen. O-03 bis O-06 und O-08 sind behoben (Abschnitt 3.3). Die übrigen Zeilen sind Restrisiken.

Nr.	Punkt	Stand und Empfehlung
O-01	Geheimnisse in der Git-History auf GitHub: `nginx/ssl/key.pem`, `frontend/.env.local`, Postman-Vault-Key, die alten Werte aus `docker-compose.yml` und das Passwort aus S-18.	**Akzeptiert (Demo-Anwendung).** JWT-Schlüssel, NextAuth-Secret und Datenbankpasswort sind dennoch bereits ersetzt. Bei einem Einsatz mit echten Daten: TLS-Zertifikat neu erzeugen und Postman-Vault-Key ersetzen. Unabhängig von der Demo-Anwendung gilt: Wird das Passwort aus S-18 an anderer Stelle verwendet, ist es dort zu ändern.
O-02	Das frühere Standardpasswort von `admin@example.com` gilt in der bestehenden Datenbank noch.	**Akzeptiert (Demo-Anwendung).** Bei einem Einsatz mit echten Daten `./scripts/reset_password.sh` ausführen.
O-07	`npm audit` meldet 5 Einträge (hoch) in Entwicklungswerkzeugen, alle über `braces` im Lint-Plugin von Next.js (`eslint-config-next`). Sie laufen nur beim Lint mit eigenen Dateien, nicht im ausgelieferten Frontend.	Für `braces` gibt es keine korrigierte Version. Beobachten und aktualisieren, sobald das Plugin ohne `braces` auskommt.
O-09	Die CSP erlaubt weiterhin Inline-Styles (`style-src 'unsafe-inline'`).	Geringes Risiko: Styles führen keinen Code aus. Eine Nonce auch für Styles wäre möglich, sobald keine Komponente mehr Inline-Styles braucht.
O-10	Die Abmeldung widerruft alle Tokens des Benutzers, auch die auf anderen Geräten. Der Sperrzähler der Kontosperre (S-22) liegt im Arbeitsspeicher und lässt sich missbrauchen, um einen Benutzer für 5 Minuten auszusperren; seit dem 10. Oktober 2026 genügen dafür drei falsche Passwörter. Ein Admin kann die Sperre in der Benutzerverwaltung aufheben. Jede Sperre eines bestehenden Benutzers löst eine E-Mail an die Benachrichtigungsadresse aus; wer Sperren mutwillig auslöst, erzeugt damit auch E-Mails (höchstens eine pro Benutzer in 5 Minuten).	Bewusst einfach gehalten. Für eine Abmeldung pro Gerät bräuchte es eine Liste einzelner Tokens.
O-11	Rollen lassen sich nur beim Anlegen eines Benutzers vergeben (`/auth/register`, `create_user.sh`). Ändern oder Löschen von Benutzern geht nur direkt in der Datenbank.	Benutzerverwaltung für Administratoren ergänzen, falls die Anwendung mehr als eine Demo wird.

5. Überprüfung

5.1 Erste Prüfung (4. Oktober 2026)

Prüfung	Ergebnis
`dotnet test`	51 von 51 bestanden
`dotnet list package --vulnerable --include-transitive`	keine anfälligen Pakete
`npm audit --omit=dev`	0 Lücken (mit Entwicklungswerkzeugen: 7, siehe O-07)
Frontend-Build mit Next.js 15 (inkl. Typprüfung und Lint) im Docker-Image	erfolgreich
`POST /auth/register` ohne Token über nginx	401
Startseite ohne Session, auch mit dem Header `x-middleware-subrequest`	307 (Umleitung zum Login)
Anmeldung über NextAuth, Startseite und Städteliste mit Session	200, Daten vorhanden
25 schnelle Anmeldeversuche über nginx	4 × 401, 21 × 429
Antwort-Header auf `/login`	alle neuen Header vorhanden, `Server: nginx` ohne Version
Veröffentlichte Ports	5000 und 5432 nur an 127.0.0.1; 80/443 offen
Benutzer in den Containern	Backend UID 1654 (`app`), Frontend `node`
Anmeldung mit dem alten Standardpasswort an der bestehenden Datenbank	weiterhin möglich (O-02, akzeptiert)

Nicht im Browser geprüft: das Verhalten des Login-Formulars bei einer manipulierten `callbackUrl` (S-09) und die Bedienung der Formulare nach dem Upgrade auf React 19. Beides ist nur durch den erfolgreichen Build und die Aufrufe per `curl` abgedeckt.

5.2 Zweite Prüfung (9. Oktober 2026)

Prüfung	Ergebnis
`dotnet test`	60 von 60 bestanden (9 neu)
`dotnet list package --vulnerable --include-transitive`	keine anfälligen Pakete
`npm audit --omit=dev`	0 Lücken (mit Entwicklungswerkzeugen: 9, siehe O-07)
Frontend-Build mit Node.js 22 (inkl. Typprüfung und Lint) im Docker-Image	erfolgreich; im Container läuft Node.js 22.23.3
`/odata/`, `/odata/$metadata`, `/odata/Adressen`, `/settings` ohne Token über nginx	401
`/odata/Adressen` und `/odata/$metadata` mit Token	200
`/odata/Cities?$top=1000` mit Token	400 mit Fehlermeldung, ohne Ausnahmetyp und Stacktrace
6 Anmeldeversuche mit falschem Passwort für dieselbe Adresse, direkt am Backend	5 × 401, danach 429

TLS 1.2 mit `AES256-SHA`, `AES256-GCM-SHA384` (statisches RSA) und `ECDHE-RSA-AES256-SHA384` (CBC)	Verbindung abgelehnt
TLS 1.2 und TLS 1.3 ohne Vorgabe	`ECDHE-RSA-AES256-GCM-SHA384` bzw. `TLS_AES_256_GCM_SHA384`
Antwort-Header	`Cache-Control: no-store` auf Backend-Antworten; neue CSP auf allen Antworten; kein `X-Powered-By`, kein `Server: Kestrel`
Anmeldung über NextAuth mit einem vorübergehend angelegten Benutzer; Startseite, Städteliste und Einstellungen mit Session	200, Städteliste mit Daten
Container-Einstellungen von Backend und Frontend	`no-new-privileges` gesetzt, alle Capabilities entfernt
`create_user.sh` mit Grossbuchstaben in der E-Mail-Adresse; `reset_password.sh` mit zu kurzem Passwort als Argument	in Kleinbuchstaben gespeichert; abgelehnt

Nicht im Browser geprüft: ob die neue Content-Security-Policy (S-28) im Browser etwas blockiert, und der Hinweis des Login-Formulars bei einer abgelehnten Anmeldung (S-25). Die Login-Seite lädt nur Dateien vom eigenen Ursprung, und alle Seiten antworten per `curl` mit 200; Verstösse gegen die CSP zeigt aber nur die Konsole des Browsers. Die Google-Anmeldung selbst liess sich nicht prüfen, weil sie nicht konfiguriert ist.

5.3 Bearbeitung der offenen Punkte (9. Oktober 2026)

Prüfung	**Ergebnis**
`dotnet test`	69 von 69 bestanden (9 neu: Rollen, Widerruf, Seitengrösse)
`dotnet list package --vulnerable --include-transitive`	keine anfälligen Pakete
`npm audit --omit=dev`	0 Lücken (mit Entwicklungswerkzeugen: 5, siehe O-07)
Browser: Anmeldung, Startseite, Städte, Einstellungen, beide Formulare	alle Seiten laden; keine Fehler und keine CSP-Verstösse in der Konsole
Browser: Aufruf einer geschützten Seite ohne Session	Umleitung zum Login, nach der Anmeldung zurück auf die gewünschte Seite
Browser: Adresse anlegen, ändern und löschen	erfolgreich, die Liste zeigt jeweils den neuen Stand
Browser: Städte suchen (PLZ, Ortsname mit Umlaut) und blättern	606 Treffer für «8» auf 25 Seiten; 25 Treffer für «zürich»; eine Eingabe mit Hochkommas ergibt 0 Treffer
Browser: `/api/auth/session`	enthält Name, E-Mail, Id und Rolle, kein Token
Browser: eingeschleuster Inline-Handler (`<img onerror=…>`)	von der CSP blockiert
Proxy `/api/backend`: ohne den eigenen Header; Pfad `/auth`; Pfad mit `..`	403; 404; 404
`GET /odata/Cities` ohne `$top`	100 Zeilen und `@odata.nextLink`
Browser: Abmelden, danach das alte Session-Cookie wieder einsetzen	das Backend weist das Token ab (401)
Browser: Benutzer mit Rolle `User` auf der Einstellungsseite	Formular gesperrt mit Hinweis; `PUT /settings` ergibt 403

Verbindung des Backends zur Datenbank	als `bandaddress_app` (kein Superuser, darf keine Datenbanken und Rollen anlegen)
Bestehende Datenbank nach dem Start	Spalten `Role` und `TokenVersion` ergänzt, vorhandene Benutzer sind `Admin`
Oberfläche vor und nach der Migration auf Tailwind CSS 4 (Bildschirmfotos)	gleiches Aussehen; die Aktionsspalte der Städteliste bricht nicht mehr um

Mit diesen Browser-Prüfungen sind auch die Punkte abgedeckt, die in 5.1 und 5.2 als «nicht im Browser geprüft» vermerkt sind, mit zwei Ausnahmen: die Google-Anmeldung (nicht konfiguriert) und der Hinweis des Login-Formulars bei einer abgelehnten Anmeldung. Geprüft wurde in einem einzigen Browser (Chromium); andere Browser und die Bedienung von Hand nicht.

6. Auswirkungen auf den Betrieb

- Der Stack braucht vor dem Start die Datei `.env`; auf diesem Rechner ist sie mit neuen Werten angelegt.
- Bestehende Sitzungen sind durch den neuen JWT-Schlüssel und das neue NextAuth-Secret ungültig; einmal neu anmelden.
- Lokales `dotnet run` braucht `Jwt__Key` und `ConnectionStrings__DefaultConnection` aus der Umgebung (siehe README).
- Neue Benutzer entstehen über `./scripts/create_user.sh` oder durch einen angemeldeten Benutzer über `/auth/register`.

Seit der zweiten Prüfung gilt zusätzlich:

- Images neu bauen (`docker compose up --build -d`) und nginx neu starten, damit die geänderte Konfiguration gilt. Bestehende Sitzungen bleiben gültig.
- Nach 3 falschen Passwörtern ist eine E-Mail-Adresse 5 Minuten gesperrt. Ein Admin hebt die Sperre in der Benutzerverwaltung auf. Ein Neustart des Backends hebt sie nur für Adressen ohne Benutzer auf.
- `/odata/$metadata` verlangt jetzt ein Token. Werkzeuge wie Postman brauchen dafür den `Authorization`-Header.
- Eine Datei von einem fremden Ursprung (Schrift, Skript, Bild, API) braucht einen passenden Eintrag in der CSP in `nginx/nginx.conf`, sonst blockiert der Browser sie.
- Die Google-Anmeldung wird abgelehnt, bis das Backend für eine Google-Identität ein Token ausstellen kann.
- Sehr alte Clients ohne ECDHE und AES-GCM können sich nicht mehr verbinden; aktuelle Browser sind nicht betroffen.

Seit der Bearbeitung der offenen Punkte gilt zusätzlich:

- `.env` braucht `APP_DB_PASSWORD`. Bei einer bestehenden Datenbank einmalig `docker compose up -d db` und `./scripts/create_db_role.sh` ausführen, danach den Stack neu bauen. Auf diesem Rechner ist das erledigt.
- Das Backend ergänzt beim Start die Spalten `Role` und `TokenVersion` in der Tabelle `Users`. Vorhandene Benutzer werden `Admin`.
- Neue Benutzer erhalten die Rolle `User`, sofern nicht `Admin` angegeben wird (`create_user.sh`: viertes Argument; `/auth/register`: Feld `role`).
- Abmelden beendet die Sitzungen des Benutzers auf allen Geräten.
- Listenabfragen liefern höchstens 100 Zeilen. Wer die API direkt nutzt, folgt `@odata.nextLink` oder blättert mit `$top` und `$skip`.
- Die Anwendung im Browser ruft die API nur noch über `/api/backend` auf. `/odata`, `/settings` und `/auth` bleiben für Werkzeuge wie Postman mit eigenem Token erreichbar.